

Array:

An array is a single variable that can store multiple values.

Normally, one variable stores one value:

```
name="Asif"
```

An array can store multiple values:

```
names=("Asif" "Rahul" "John")
```

Why Do We Use Arrays?

Suppose you have three servers.

Without an array:

```
server1="web1"
```

```
server2="web2"
```

```
server3="web3"
```

You have three separate variables.

With an array:

```
servers=("web1" "web2" "web3")
```

Type	Meaning	Example
Indexed Array	Uses numbers as indexes	<code>servers=("web1" "web2")</code>
Associative Array	Uses names/keys	<code>server_ip[web1]="10.0.0.1"</code>

Indexed Array:

An indexed array stores multiple values using numeric indexes starting from 0.

Symbol / Keyword	Purpose	Example
------------------	---------	---------

()	Creates an array	servers=("web1" "web2")
\${ }	Accesses/expands the array	\${servers[0]}
[0]	Accesses first value	\${servers[0]}
[1]	Accesses second value	\${servers[1]}
[2]	Accesses third value	\${servers[2]}
[@]	Gets all values	\${servers[@]}

Associative Array:

An **associative array** stores values using **names/keys instead of numbers**.

Use:

declare -A

Example:

declare -A servers

servers[web1]="10.0.0.1"

servers[web2]="10.0.0.2"

Symbol / Keyword	Purpose	Example
declare -A	Creates an associative array	declare -A servers
[key]	Defines/accesses a named key	servers[web1]="10.0.0.1"
\${ }	Expands the array value	\${servers[web1]}
[@]	Gets all values	\${servers[@]}

Declaring Arrays:

declare -a servers

servers=("web1" "web2" "web3")

Keyword	Purpose	Example
---------	---------	---------

<code>declare</code>	Declares a variable/array	<code>declare -a servers</code>
<code>-a</code>	Specifies an indexed array	<code>declare -a servers</code>
<code>servers</code>	Array name	<code>declare -a servers</code>

Accessing Array Elements:

Accessing array elements means retrieving or getting a specific value from an array by using its index.

Symbol / Keyword	Purpose	Example
<code>\${}</code>	Expands/gets the value of a variable or array element	<code>\${servers[0]}</code>
<code>[0]</code>	Accesses the first element	<code>\${servers[0]}</code>
<code>[1]</code>	Accesses the second element	<code>\${servers[1]}</code>
<code>[2]</code>	Accesses the third element	<code>\${servers[2]}</code>
<code>[@]</code>	Gets all elements of the array	<code>\${servers[@]}</code>
<code>[*]</code>	Gets all elements of the array	<code>\${servers[*]}</code>

Looping Through Arrays:

Looping through an array means going through each element of the array one by one and performing an action on each value.

Basic Example

First, create an array:

```
servers=("web1" "web2" "web3")
```

Now loop through it:

```
for server in "${servers[@]}"
do
    echo "$server"
done
```

Output

web1

web2

web3

Symbol / Keyword	Purpose	Example
for	Starts the loop	for server in ...
in	Takes values from the array	in "\${servers[@]}"
\${servers[@]}	Gets all array elements	"\${servers[@]}"
do	Starts the loop body	do
done	Ends the loop	done
\$server	Gets the current element	echo "\$server"

Modifying Arrays:

Modifying an array means changing the values stored in an array.

We can add, change, or remove elements from an array.

Symbol / Keyword	Purpose	Example
[index]=value	Changes or adds a value at a specific index	servers[1]="db1"
+=	Adds a new element	servers+=("web4")
unset	Removes an element	unset 'servers[1]'
\${servers[@]}	Displays all elements	echo "\${servers[@]}"
\${#servers[@]}	Gets number of elements	echo "\${#servers[@]}"

Slicing and Subsets:

Array slicing means selecting a specific portion (subset) of an array instead of accessing the whole array.

Example:

```
servers=("web1" "web2" "web3" "web4" "web5")
```

```
echo "${servers[@]:1:3}"
```

Result:

```
web2 web3 web4
```

Symbol / Keyword	Purpose	Example
<code>\${}</code>	Expands the array	<code>\${servers[@]}</code>
<code>[@]</code>	Selects all array elements	<code>\${servers[@]}</code>
<code>:</code>	Starts the slicing operation	<code>\${servers[@]:1:3}</code>
start	Starting index	<code>\${servers[@]:1:3}</code>
length	Number of elements to select	<code>\${servers[@]:1:3}</code>

Exit Codes with Arrays:

An exit code is a number returned by a command or script to tell whether it succeeded or failed.

Symbol / Keyword	Purpose	Example
<code>\$?</code>	Gets exit code of previous command	<code>echo "\$?"</code>
<code>0</code>	Indicates success	<code>exit 0</code>
<code>1</code>	Commonly used to indicate failure	<code>exit 1</code>
<code>for</code>	Processes array elements one by one	<code>for service in "\${services[@]}"</code>
<code>exit</code>	Ends a script with an exit code	<code>exit 1</code>
<code>\${array[@]}</code>	Gets all array elements	<code>"\${services[@]}"</code>

`$?` → previous command's exit code

`0` → success

non-zero → failure/error

`exit 0` → script completed successfully

User Input:

Taking information from the person running the Bash script and storing it in a variable.

Basic Example

```
#!/bin/bash
```

```
echo "Enter your name:"  
read name
```

```
echo "Hello $name"
```

When you run it:

Enter your name:

Asif

Hello Asif

Symbol / Keyword	Purpose	Example
read	Takes input from the user	read name
\$	Gets the value of the variable	echo "\$name"
=	Assigns a value	name="Asif"
-p	Displays a prompt while taking input	read -p "Enter name:" name
-s	Hides input while typing	read -s password
-r	Prevents backslash from being treated specially	read -r name

read → take user input

read name → input stored in name

\$ name → access its value as \$name

read -p → input with prompt

read -s → hidden input

Script Arguments:

Script arguments are values passed to a Bash script from the command line when the script is executed.

Create `greet.sh`:

```
#!/bin/bash
```

```
echo "Name: $1"
```

```
echo "City: $2"
```

Run:

```
./greet.sh Asif Hyderabad
```

Output:

```
Name: Asif
```

```
City: Hyderabad
```

Symbol / Keyword	Purpose	Example
\$1	First argument	echo "\$1"
\$2	Second argument	echo "\$2"
\$3	Third argument	echo "\$3"
\$n	nth argument	echo "\$n"

Accessing Script Arguments:

suppose you run:

```
./script.sh Asif DevOps Hyderabad
```

Then:

\$1 → Asif
\$2 → DevOps
\$3 → Hyderabad

Example:

```
#!/bin/bash
```

```
echo "Name: $1"  
echo "Course: $2"  
echo "Location: $3"
```

Output:

```
Name: Asif  
Course: DevOps  
Location: Hyderabad
```

Special Variables in Bash :

Special variables are predefined Bash variables that provide information about the script, its arguments, processes, and command status.

Special Variable	Purpose	Example
\$0	Script name	echo "\$0"
\$#	Number of arguments	echo "\$#"
\$@	All arguments separately	echo "\$@"
\$*	All arguments as one group	echo "\$*"
\$?	Exit status of last command	echo "\$?"
\$\$	Current shell's process ID	echo "\$\$"
\$_	PID of last background command	echo "\$_"

<code>\$-</code>	Current shell flags/options	echo "\$-"
<code>\$_</code>	Last argument of previous command	echo "\$_"

`$0` — Script Name:

`$0` contains the name/path used to run the script.

Example:

```
#!/bin/bash
```

```
echo "Script name: $0"
```

Run:

```
./test.sh
```

Output:

```
Script name: ./test.sh
```

Script Arguments:

Variable	Purpose	Example
<code>\$0</code>	Script name	echo "\$0"
<code>\$1, \$2...</code>	Script arguments	echo "\$1"
<code>\$#</code>	Number of arguments	echo "\$#"
<code>\$@</code>	All arguments separately	echo "\$@"
<code>\$*</code>	All arguments	echo "\$*"
<code>\$?</code>	Last command's exit status	echo "\$?"

<code>\$\$</code>	Current shell/process PID	<code>echo "\$\$"</code>
<code>\$!</code>	Last background process PID	<code>echo "\$!"</code>
<code>\$-</code>	Current shell flags	<code>echo "\$-"</code>
<code>\$_</code>	Last argument of previous command	<code>echo "\$_"</code>

Easy Memory Trick

`$0` → Script name
`$1` → First argument
`$2` → Second argument
`$#` → How many?
`$@` → All arguments
`$*` → All together
`$?` → Success or failure?
`$$` → My PID
`$!` → Background PID
`$-` → Shell flags
`$_` → Last argument

Output Redirection:

Output redirection means sending the output of a command to a file instead of displaying it on the screen.

Symbol	Purpose	Example
<code>></code>	Writes output to a file; overwrites existing content	<code>ls > files.txt</code>
<code>>></code>	Adds output to the end of a file	<code>ls >> files.txt</code>

Example: >

```
echo "Hello Asif" > message.txt
```

Instead of showing:

```
Hello Asif
```

on the screen, it puts it inside `message.txt`.

⚠ If `message.txt` already has content, `>` **overwrites it**.

Example: >>

```
echo "Welcome to DevOps" >> message.txt
```

This **adds** the text to the existing file.

Easy memory

> → Create/overwrite

>> → Add/append

Input Redirection:

Input redirection means taking input for a command from a file instead of from the keyboard.

Example

Suppose `names.txt` contains:

```
Asif  
Rahul  
John
```

You can use:

```
wc -l < names.txt
```

The `wc` command receives its input from `names.txt`.

Output:

3

Symbol	Purpose	Example
<	Takes input from a file	wc -l < names.txt

Pipes:

A pipe sends the output of one command directly as the input of another command.

Symbol	Purpose	Example
		Sends output of one command to another command

Symbol	Purpose	Example
<	Redirect standard input	command < file.txt
>	Redirect standard output	command > output.txt
>>	Append standard output	command >> output.txt
`	`	Send output to another command
2>	Redirect error output	command 2> error.txt
2>>	Append error output	command 2>> error.txt

Very Simple Way to Remember

> → Send output TO a file

>> → Add output TO a file

< → Take input FROM a file

| → Send output TO another command

Exit Status and Error Codes:

Every command in Bash returns a number called an exit status. It tells us whether the command succeeded or failed.

The most important rule:

0 → Success
1-255 → Failure / Error

Capturing Exit Status:

Capturing exit status means storing the result of a command so that we can check it later.

`$?` contains the exit status of the **most recently executed command**.

Symbol / Keyword	Purpose	Example
<code>\$?</code>	Gets last command's exit status	<code>echo "\$?"</code>
<code>=</code>	Stores the status in a variable	<code>status=\$?</code>
0	Success	<code>if ["\$status" -eq 0]</code>
Non-zero	Failure	<code>if ["\$status" -ne 0]</code>

Common Exit Codes:

Exit Code	Common Meaning	Example
0	Success	Command completed successfully
1	General error	General failure
2	Incorrect command usage / syntax in many commands	Invalid usage
126	Command found but cannot be executed	Permission problem
127	Command not found	<code>abc command</code>
128	Invalid exit argument in Bash	<code>exit 128</code>
130	Command terminated by Ctrl+C	SIGINT

Exit Status in Conditional Statements:

`if ls /home`

`then`

```
    echo "Command succeeded"
else
    echo "Command failed"
Fi
```

```
ls succeeds → 0 → then
ls fails    → non-zero → else
```

Using \$? in an if:

```
systemctl is-active --quiet nginx
status=$?
```

```
if [ "$status" -eq 0 ]
then
    echo "Nginx is running"
else
    echo "Nginx is not running"
Fi
```

Exit Status for Functions:

A Bash function also returns an exit status.

```
check_service() {
    systemctl is-active --quiet nginx
}
```

```
check_service
echo "$?"
```

If nginx is active:

0

Exit Status for Commands in Pipelines:

Example:

```
ls | grep ".txt"
```

Normally, the pipeline's status is based on the **last command** in the pipeline.

Example:

```
echo "hello" | grep "hello"
```

```
echo "$?"
```

Output:

0

Exit Codes in Scripts:

A Bash script can return an exit code to the operating system or another program when it finishes.

Example

```
#!/bin/bash
```

```
if [ -f "/etc/nginx/nginx.conf" ]
then
    echo "Nginx configuration exists"
    exit 0
else
    echo "Nginx configuration not found"
    exit 1
fi
```

Run:

```
./check_nginx.sh
```

Then check:

echo "\$?"

If the file exists:

0

If it doesn't:

1

\$? → What was the result?

0 → Success

non-zero → Failure/status condition

return → Function result

exit → Script result

pipefail → Check failures in pipelines

Topic	Purpose	Example
Exit Status	Shows command result	echo "\$?"
Capturing Status	Store result for later	status=\$?
Conditional Status	Check success/failure	if [\$? -eq 0]
Function Status	Return function result	return 0
Pipeline Status	Check command chain	cmd1 cmd2
Script Exit Code	Tell caller script result	exit 0

Symbol / Keyword	Purpose	Example
\$?	Gets last command exit status	echo "\$?"
0	Indicates success	exit 0
1	Indicates general failure	exit 1
\$#	Number of script arguments	echo "\$#"
return 0	Function succeeded	return 0
return 1	Function failed	return 1
exit 0	Script completed successfully	exit 0

<code>exit 1</code>	Script completed with failure	<code>exit 1</code>
<code>set -o pipefail</code>	Detects pipeline failures	<code>set -o pipefail</code>